

Análise de Código Java

Classe: `ImmutableConfig` · Padrão: Builder + Imutabilidade

Java OOP

Imutabilidade

Builder Pattern

Defensive Copy

1. Objetivo Geral

A classe `ImmutableConfig` representa um objeto de configuração de rede (host, porta, timeout e SSL) que, uma vez criado, não pode ser modificado. Toda alteração produz uma *nova instância*, preservando a integridade do objeto original. O método `main` demonstra a criação de configurações e a geração de variantes sem alterar o estado existente.

2. Lógica e Algoritmo Central

A lógica central combina dois padrões: o **Builder Pattern** para construção fluente e segura, e a **cópia defensiva** para modificação sem mutação. O método `withPort` exemplifica o segundo: ele cria um novo Builder inicializado com o estado atual e apenas substitui a porta, retornando uma instância completamente nova. O construtor privado garante que nenhum código externo bypass o Builder.

```
public ImmutableConfig withPort(int newPort) {  
    return new Builder(this).port(newPort).build();  
}
```

3. Conceitos e Recursos Java Envolvidos

Campos final	Todos os atributos são declarados com final, impedindo reatribuição após a construção.
Construtor privado	Força o uso exclusivo do Builder, controlando como os objetos são instanciados.
Objects.requireNonNull	Valida o campo obrigatório host em tempo de construção, lançando NullPointerException com mensagem clara.
Fluent Interface	Cada método do Builder retorna this, permitindo encadeamento: .host(...).port(...).ssl(...).build().
Static Nested Class	O Builder é uma classe estática aninhada, acoplada semanticamente mas independente de instância.

4. Complexidade e Desempenho

Tempo: $O(1)$ para criação e acesso — todas as operações são de atribuição direta ou retorno de campo. O método `withPort` cria um novo objeto em tempo constante.

Espaço: $O(1)$ por instância — o tamanho do objeto é fixo (4 campos primitivos/String). Em cenários com muitas variações, o custo de alocação pode ser relevante, pois cada chamada a `withPort` gera um novo objeto no heap, sujeito à coleta de lixo.

5. Decisões de Design e Limitações

Decisão positiva: A imutabilidade torna o objeto intrinsecamente *thread-safe* — múltiplas threads podem compartilhar a mesma instância sem risco de condição de corrida, sem necessidade de sincronização. Além disso, objetos imutáveis são ideais como chaves de mapas ou constantes de configuração.

Limitação: O campo `timeout` não possui getter público, tornando-o inacessível fora da classe. O Builder também não valida o intervalo da porta (ex.: 0–65535) nem o valor do timeout, o que pode permitir configurações inválidas em tempo de execução. Para sistemas críticos, recomenda-se adicionar validações explícitas no método **`build()`**.